Kubernetes x NVIDIA DRA Workshop

Dynamic Resource Allocation for GPUs

Yang Tzu-Ching (Shiritai)

Environment: Kubernetes 1.34 · NVIDIA A100 / RTX4090 / RTX5090 · Kind Cluster

Demo scripts

```
git clone https://github.com/Shiritai/k8s-dra-workshop
```

About Me

Yang Tzu-Ching 楊子慶

GitHub: Shiritai

- **Open Source Developer**: `sanity-gravity`, `xc-tauri-rasp`, `openadkit`, `xv6-ntu-mp`, ...
- Kubernetes & GPU infrastructure enthusiast

Agenda

#	Topic	Modules	Style
1	From `docker run --gpus` to DRA	—	Lecture
2	K8s Basics for Docker Engineers	—	Lecture
3	DRA Core: DeviceClass, Claim, Slice	**M0–M3**	Demo
4	CDI: How Hardware Gets Into Containers	—	Lecture
5	GPU Sharing with MPS	**M4–M5**	Demo
5½	MPS in Practice: vLLM Inference	**M6**	Demo
6	Hardware Isolation with MIG	**M10.1–10.3**	Demo
7	MIG x MPS Hybrid + Traceability	**M10.4–10.5**	Demo
8	Production: Capacity, Admin, Resilience	**M7–M9**	Demo
9	Architecture Decisions & Wrap-up	—	Discussion

The Starting Point

What You Already Know

```
docker run --gpus all nvidia/cuda:12.3.1-base-ubuntu22.04 nvidia-smi
```

This gives you **all GPUs**. Simple. Direct. Done.

But in production, you need answers to harder questions:

- Can two teams **share** one GPU without interference?
- Can I request *"a GPU with ≥ 20GB memory"* instead of *"GPU 0"*?
- Can multiple containers **automatically restart** if they crash?
- Can one container's crash be **guaranteed** not to affect another?

The Problem with Device Plugin

The Old Way: Blind Integer Counting

```
# K8s Device Plugin (the old way)
resources: { limits: { nvidia.com/gpu: 1 } }  # "Give me 1 GPU" — which one? ¯\_(ツ)_/¯
```

What the Scheduler sees: *"Node A has 4 GPUs left"*
What the Scheduler doesn't see: model, memory, architecture, MIG slices

The Scheduler is **blind**. Kubelet picks a random GPU at the last moment.

> No sharing. No filtering. No hardware awareness.
> This is the `void *gpu = malloc(sizeof(GPU))` of GPU scheduling — you get a pointer, that's it.

DRA: The New Way

Structured, Declarative GPU Allocation (GA in K8s 1.34)

```yaml
apiVersion: resource.k8s.io/v1
kind: ResourceClaim
metadata: { name: my-gpu }
spec:
  devices:
    requests:
    - name: gpu-req
      exactly:
```

```yaml
deviceClassName: gpu.nvidia.com
selectors:
  - cel:
    expression: "device.capacity['gpu.nvidia.com']
                 .memory
                 .compareTo(quantity('30Gi'))
                >= 0"
```

Scheduler reads this → matches against hardware snapshots → picks the best device **before** Pod starts. This is the `auto gpu = make_unique<GPU>(VRAM >= 30_GiB)` of GPU scheduling. Nice!

Sum up: Docker → K8s

Docker (Imperative)

```
docker run \
  --gpus all \
  my-image python train.py
```

You **tell** Docker what to do.
If it crashes, it stays crashed. 👆

You **describe** what you want → K8s **maintains** that state continuously. 👉

Kubernetes (Declarative)

```yaml
apiVersion: v1
kind: Pod
spec:
  containers:
  - name: train
    image: my-image
    command: ["python", "train.py"]
    resources:
      claims: [{name: gpu}]
  resourceClaims:
  - name: gpu
    resourceClaimName: my-claim
```

K8s DRA Architecture

```
┌──────────────────────────────────────┐
│         CONTROL PLANE (Brain)        │
│   ┌──────────┐   ┌────────────────┐  │
│   │   API    │   │   Scheduler    │  │   ResourceClaim ──> "I need a GPU with SM ≥ 50"
│   │  Server  │   │ (reads Slice,  │  │   ResourceSlice ──> "Node-A has 3g.20gb + 4g.20gb"
│   └──────────┘   │ matches Claim) │  │
│                  └────────┬───────┘  │
└──────────────────────────┼──────────┘
                           │ allocation decision
┌──────────────────────────V──────────┐
│         DATA PLANE (Muscle)          │
│     ┌──────────────────────┐         │
│     │  Kubelet + DRA Plugin  │──────> Generate CDI spec → inject GPU into container
│     └──────────────────────┘         │
└──────────────────────────────────────┘
```

Key DRA change: Scheduler is no longer blind — it reads *ResourceSlice* hardware snapshots.

The Three DRA Objects

Object	Scope	Created By	Purpose
DeviceClass	Cluster	Admin / Driver	*"What counts as a GPU? What counts as MIG?"*
ResourceSlice	Cluster	DRA Plugin	*"Node-A has: gpu-0 (A100), gpu-1-mig-0-0-8 (7g.40gb)"*
ResourceClaim	Namespace	Developer	*"I need a MIG instance with ≥ 50 SMs"*

Docker Analogy

- `DeviceClass` ≈ the `--gpus` flag definition (what devices are available)
- `ResourceSlice` ≈ `nvidia-smi` output (what hardware exists)
- `ResourceClaim` ≈ `docker run --gpus device=0` (request specific device)

> The difference: all three are **standard K8s API objects**, RBAC-protected and declaratively managed.

DeviceClass: The Filter Contract

```yaml
apiVersion: resource.k8s.io/v1
kind: DeviceClass
metadata:
  name: gpu.nvidia.com         # Full GPU (not MIG)
spec:
  selectors:
  - cel:
      expression: >-
        device.driver == 'gpu.nvidia.com' &&
        device.attributes['gpu.nvidia.com'].type == 'gpu'
```

```yaml
apiVersion: resource.k8s.io/v1
kind: DeviceClass
metadata:
  name: mig.nvidia.com         # MIG instances only
spec:
  selectors:
  - cel:
      expression: >-
        device.driver == 'gpu.nvidia.com' &&
        device.attributes['gpu.nvidia.com'].type == 'mig'
```

These are auto-created by the NVIDIA DRA Driver Helm chart.

⚡ Environment Setup: Prerequisites (Module 0)

Required Tools

Tool	Purpose	Verify
NVIDIA Driver	GPU driver	`nvidia-smi`
Docker	Container runtime	`docker ps`
Kind	K8s cluster	`kind version`
Helm	DRA Driver install	`helm version`
nvidia-ctk	CDI config	`nvidia-ctk cdi list`

One-command check

```
./scripts/phase1/run-module0-check-env.sh

✅ NVIDIA Driver detected.
✅ Docker detected.
✅ ...
✅ NVIDIA CTK CDI is configured.
```

⚠️ If NVIDIA runtime is missing:

```
sudo nvidia-ctk runtime configure \
  --runtime=docker
sudo systemctl restart docker
```

⚡ Live Demo: First DRA Pod (Modules 1–3)

Module 1: Kind Cluster

```
./scripts/phase1/run-module1-setup-kind.sh
```

Module 2: DRA Driver

```
./scripts/phase1/run-module2-install-driver.sh
```

Module 3: First GPU Pod

```
./scripts/phase1/run-module3-verify-workload.sh
```

What to watch for:

1. Kind node can see GPUs via `nvidia-smi`
2. ResourceSlice published → `kubectl get resourceslices`
3. Pod gets GPU through **ResourceClaim**, not Device Plugin

Architecture flow

```
Host GPU → Kind Node (CDI passthrough)
  → DRA Driver (Helm) → ResourceSlice
    → ResourceClaim → Pod gets GPU
```

The Claim → Schedule → Prepare → Run Flow

```
1. You create ResourceClaim          "I want a GPU"
          ↓
2. Scheduler reads ResourceSlice     "Node has gpu-0 (A100, 40GB, SM 8.0)"
          ↓
3. Scheduler matches & allocates     "gpu-0 → this Claim"
          ↓
4. Kubelet calls DRA Plugin          "Prepare gpu-0 for this Pod"
          ↓
5. Plugin generates CDI spec         /var/run/cdi/k8s.gpu.nvidia.com-device_base.yaml
          ↓
6. containerd reads CDI, injects     /dev/nvidia0, drivers, env vars
          ↓
7. Container starts with GPU         nvidia-smi works! ✅
```

CDI: The Bridge Between K8s and Hardware

Container Device Interface — the "translator"

Docker way: Runtime hardcodes `--gpus` handling
DRA way: Driver produces a standard CDI spec → any runtime can consume it

```yaml
# /var/run/cdi/k8s.gpu.nvidia.com-device_base.yaml (auto-generated)
devices:
  - name: "gpu-0-mig-5-0-4"          # DRA device ID
    containerEdits:
      deviceNodes:
        - path: "/dev/nvidia0"        # Parent GPU
        - path: "/dev/nvidia-caps/nvidia-cap21"   # MIG-specific caps
        - path: "/dev/nvidia-caps/nvidia-cap22"
```

CDI: The Bridge Between K8s and Hardware (Cont.)

Container Device Interface — the "translator"

Two CDI locations:

Path	Purpose	Generated By
`/etc/cdi/nvidia.yaml`	Static — all GPUs for Kind passthrough	`nvidia-ctk` on host
`/var/run/cdi/*.yaml`	Dynamic — per-device, DRA-controlled	DRA Plugin

GPU Sharing: The Problem

One A100 costs $15K+. Most workloads don't need a full card.

Without sharing:

```
Pod A: 100% of A100     ← uses 20% actual compute
Pod B: Pending...       ← waiting for another GPU
Pod C: Pending...
```

With sharing:

```
Pod A: 33% of A100      ← gets what it needs
Pod B: 33% of A100      ← runs immediately
Pod C: 33% of A100      ← everyone's happy
```

Sharing Technologies	MPS (Software)	MIG (Hardware)
Mechanism	Aggregate CUDA contexts	Physical partition
Isolation	Soft (cuda limits)	Hard (separate memory controller)
OOM blast radius	With memory limit / Noisy neighbor	**Impossible** (hardware isolation)
GPU support	All NVIDIA GPUs	A100 / H100 only

MPS: How It Works

Multi-Process Service = Spatial Sharing

```
Without MPS:                          With MPS:

┌──────────┐   ┌──────────┐          ┌──────────┐   ┌──────────┐
│ Pod A    │   │ Pod B    │          │ Pod A    │   │ Pod B    │
└──────────┘   └──────────┘          └──────────┘   └──────────┘
     │              │                     └──────────┬──────────┘
     V              V                                V
   GPU <─── context switch                      ┌──────────┐
   (expensive, serial)                          │ MPS Server │
                                                └──────────┘
                                                       V
                                                GPU (concurrent kernels)
```

MPS Server aggregates CUDA commands → **kernels run simultaneously** on the GPU.

MPS in DRA: Declare in ResourceClaim

The Driver handles everything (Module 4)

ResourceClaim

```yaml
config:
- opaque:
    driver: gpu.nvidia.com
    parameters:
      apiVersion: resource.nvidia.com/v1beta1
      kind: GpuConfig
      sharing: { strategy: MPS }
```

Pod spec (nothing special)

```yaml
spec:
  containers:
  - name: cuda-container
    image: nvidia/cuda:12.3.1-devel-ubuntu22.04
    resources:
      claims: [{name: gpu}]
  resourceClaims:
  - name: gpu
    resourceClaimName: gpu-claim-dra-mps
```

MPS in DRA: Declare in ResourceClaim (Cont.)

The Driver handles everything (Module 4)

No `hostIPC`. No `hostPath`. No manual env vars. The Driver automatically:

- Creates a **per-Claim MPS daemon** Deployment
- Injects MPS pipe via CDI (`/tmp/nvidia-mps`)
- Mounts **isolated** **`/dev/shm`** **tmpfs** (not the Node's IPC namespace)

MPS Resource Constraints

Prevent one Pod from killing everyone else

Environment Variable	Controls	Example
`CUDA_MPS_ACTIVE_THREAD_PERCENTAGE`	SM compute cap	`50` → max 50% of SMs
`CUDA_MPS_PINNED_DEVICE_MEM_LIMIT`	Memory hard limit	`0=4G` → max 4GB on GPU 0

Module 5: Limits in GpuConfig (driver-managed)

```yaml
sharing:
  strategy: MPS
  mpsConfig:
    defaultActiveThreadPercentage: 20        # SM cap
    defaultPinnedDeviceMemoryLimit: 1Gi      # Memory cap
```

MPS Resource Constraints (Cont.)

Prevent one Pod from killing everyone else

Users **cannot override** these limits — they're set in the Claim, not the Pod.

Real impact on vLLM (Qwen 2.5-1.5B on RTX4090):

SM Limit	Throughput	First Token Latency
100%	3.57 req/s	17 ms
40%	3.56 req/s	25 ms
20%	3.47 req/s	**43 ms** (2.5x)

⚡ Live Demo: MPS Sharing

Modules 4 → 5

```
# M4: DRA-managed MPS — 2 Pods sharing 1 GPU, zero hostIPC
./scripts/phase1/run-module4-mps-basics.sh

# M5: DRA-managed MPS with GpuConfig resource limits (driver-enforced)
./scripts/phase1/run-module5-mps-advanced.sh
```

What to watch for:

1. M4: Both Pods share the same GPU UUID, `hostIPC` is `false`
2. M5: Memory allocation beyond 1Gi limit is rejected by MPS daemon

⚡ Live Demo: vLLM on MPS (Module 6)

Real-world inference under MPS throttling

```
# M6: Deploy vLLM (Qwen2.5-1.5B) with DRA-managed MPS, verify inference
./scripts/phase1/run-module6-vllm-verify.sh

# M6: Benchmark throughput & latency under different SM limits
./scripts/phase1/run-module6-vllm-experiment.sh
```

What to watch for:

1. vLLM serves on a **shared** GPU via MPS — no `hostIPC`, no manual env vars
2. Throughput is stable across SM limits; **first-token latency** scales with SM cap
3. At 20% SM → latency ~2.5x; throughput barely drops (memory-bandwidth bound)

MIG: Hardware-Level Isolation

```
MPS (Software):                                  MIG (Hardware):

┌──────────────────────────┐                     ┌──────────────┬──────────────┐
│    Shared SM / L2 / MC   │                     │ SM │ L2│ MC│ SM │ L2│ MC│
│  ┌──────┐    ┌──────┐    │                     │  3g.20gb     │  4g.20gb     │
│  │Pod A│    │Pod B│    │                     │  ┌──────┐    │  ┌──────┐    │
│  └──────┘    └──────┘    │                     │  │Pod A│    │  │Pod B│    │
│  Software limits only    │                     │  └──────┘    │  └──────┘    │
└──────────────────────────┘                     └──────────────┴──────────────┘

Pod B OOM → Pod A safe (with limit)              Pod B OOM → Pod A: "what OOM?"
Pod B alloc all → Pod A starved (no limit)             (hardware isolation)
```

Each MIG instance has its own:

- **Streaming Multiprocessors** (dedicated compute)
- **Memory Controller** (dedicated VRAM bus), **L2 Cache** (no sharing, no interference)

A100-PCIE-40GB MIG Profiles

Profile	VRAM	SMs	Use Case
7g.40gb	40320 MiB	98	Full-card equivalent, large models
4g.20gb	20096 MiB	56	Medium models, SLA workloads
3g.20gb	20096 MiB	42	Standard inference
2g.10gb	9984 MiB	28	Light inference
1g.5gb	4864 MiB	14	Minimum granularity

Our workshop environment (3 independent MIG devices total):

```
GPU 0 (MIG enabled):  3g.20gb (42 SM) + 4g.20gb (56 SM)
GPU 1 (MIG enabled):  7g.40gb (98 SM)
```

MIG + DRA: Three Selection Strategies

1. Any MIG (zero knowledge)

```
# Just give me any MIG slice
deviceClassName: mig.nvidia.com
```

2. By profile name (explicit)

```
deviceClassName: mig.nvidia.com
selectors:
- cel:
  expression: "device.attributes['gpu.nvidia.com']
                      .profile == '7g.40gb'"
```

3. By hardware specs

```
deviceClassName: mig.nvidia.com
selectors:
- cel:
    expression: >-
      device.capacity['gpu.nvidia.com'].multiprocessors
        .compareTo(quantity('50')) >= 0 &&
      device.capacity['gpu.nvidia.com'].multiprocessors
        .compareTo(quantity('60')) < 0
```

Strategy 3 works unchanged on A100 → H100 migration. No profile name dependency.

⚡ Live Demo: MIG Selection + Isolation

Modules 10.1 → 10.3

```
# 10.1: Profile-based MIG selection (4 cases: any, 7g, 3g, 4g)
./scripts/phase3/module10-1/run.sh
# 10.2: Capacity-based CEL matching (SM ranges, VRAM thresholds)
./scripts/phase3/module10-2/run.sh
# 10.3: OOM isolation proof — crash one slice, other survives
./scripts/phase3/module10-3/run.sh
```

Module 10.3 — the money shot:

- Pod A on 4g.20gb (high SLA)
- Pod B on 3g.20gb → force 25GB allocation (OOM!)
- Pod A: still Running ✅ — **hardware isolation confirmed**

MIG × MPS: Two-Layer Architecture (Module 10.4)

Hardware isolation + software sharing = best of both worlds

```
A100 GPU
├── MIG Instance 1 (4g.20gb) ── MPS Daemon A
│   ├── Pod 1 (shared via MPS)
│   └── Pod 2 (shared via MPS)
└── MIG Instance 2 (3g.20gb) ── MPS Daemon B  ← independent daemon
    ├── Pod 3 (shared via MPS)
    └── Pod 4 (shared via MPS)
```

- **Between MIG slices:** hardware isolation (OOM can't spread)
- **Within each slice:** MPS sharing (maximize utilization)
- Each MIG slice gets its **own MPS daemon** (independent failure domains)

Silicon-to-Pod Traceability

Module 10.5: From physical hardware to K8s workload

```
Physical MIG UUID                DRA Device ID          Pod Owner
──────────────────────────────────────────────────────────────────

MIG-9ac89393-4759-5270...  →  gpu-1-mig-0-0-8    →  m10-5-pod-a
MIG-2989dad9-264a-5e4d...  →  gpu-0-mig-5-0-4    →  m10-5-pod-b
MIG-e9c20b15-3db0-52c6...  →  gpu-0-mig-9-4-4    →  m10-5-pod-c
```

Use case: Hardware alert on `MIG-9ac89393` → instantly find the affected Pod and team.

```
# ResourceSlice: physical UUID → DRA device ID
kubectl get resourceslices -o json | jq '...'
# ResourceClaim: DRA device ID → Pod owner
kubectl get resourceclaim <name> -o jsonpath='{.status.reservedFor}'
```

⚡ Live Demo: Hybrid + Traceability

Modules 10.4 → 10.5

```
# 10.4: MIG × MPS hybrid — 4 Pods on 2 MIG slices with MPS sharing
./scripts/phase3/module10-4/run.sh

# 10.5: Silicon-to-Pod traceability audit
./scripts/phase3/module10-5/run.sh
```

What to watch for:

- 10.4: Two pairs of Pods, each pair sharing the same MIG UUID
- 10.4: The two pairs use **different** MIG UUIDs (hardware isolation)
- 10.5: Complete UUID → DeviceID → Pod mapping table

Production Readiness

Three questions before going live

We've built the GPU orchestration layer. Now three critical questions remain:

Question	Module	Answer
Can the Scheduler understand **shared capacity**?	M7 (Alpha)	Not yet (driver limitation)
Can admins **debug** a fully-allocated node?	M8 (Beta)	Yes — `adminAccess: true`
Does a **driver crash** kill running workloads?	M9	No — CDI decoupling

> These are the "Day 2" questions that separate a demo from production.

Module 7: Consumable Capacity Alpha

Can the Scheduler count shared GPU memory?

```yaml
# ResourceClaim with capacity request (K8s 1.34 API)
spec:
  devices:
    requests:
    - name: req-1
      exactly:
        deviceClassName: gpu-capacity.nvidia.com
        capacity: { requests: { memory: 4Gi } }    # "I only need 4GB"
```

Theory: Scheduler deducts 4GB from gpu-0's total, leaving room for more Pods.

Reality: All Pods after the first one → `Pending`.

Module 7: The Blind Scheduler Problem Alpha

Why? Opaque Parameters = Black Box to Scheduler

```
NVIDIA Driver enforces:
  config.Opaque == nil → error("only opaque parameters are supported")
Scheduler sees:
  ResourceSlice: devices: [{name: gpu-0}]    ← 1 item = 1 GPU
  pod-small binds gpu-0 → 1/1 Used
  pod-4gi requests     → 0/1 Available → Pending ✗
```

The `capacity.requests` field exists in the API, but the **NVIDIA Driver** still uses opaque parameters. The Scheduler cannot read inside the opaque blob → falls back to **counting mode** (1 GPU = 1 item).

Status: K8s 1.34 API is ready. Waiting for NVIDIA Driver to adopt Structured Parameters.

⚡ Live Demo: Consumable Capacity

Module 7

```
./scripts/phase2/run-module7-consumable-capacity.sh
```

What to watch for:

1. `pod-small` (4Gi) → Running ✅
2. `pod-4gi` (4Gi) → Pending ❌ (Opaque 1-to-1 mapping)
3. `pod-18gi` (18Gi) → Pending ❌ (same reason)
4. `pod-overflow` (50Gi) → Pending ❌ (exceeds capacity regardless)

All three Pending pods fail for the same reason: **the Scheduler sees "0 devices available"**, not "insufficient memory".

Module 8: Admin Access Beta

The Exclusivity Problem

```
Scenario:
  GPU 0 → pod-gpu-1 (Exclusive)     ← researcher's LLM training
  GPU 0 → pod-gpu-2                 ← Pending (no available devices)
  GPU 0 → pod-admin (adminAccess)   ← Running ✅ (bypasses exclusivity)
```

Module 8: Admin Access (Cont.) Beta

The Exclusivity Problem

Two-step setup:

1. Label the namespace:

```
kubectl label namespace default \
  resource.kubernetes.io/admin-access=true
```

2. Create an admin ResourceClaim:

```yaml
spec:
  devices:
    requests:
    - name: req-1
      exactly:
        deviceClassName: gpu-admin.nvidia.com
        adminAccess: true    # Bypasses exclusivity lock
```

Module 8: Observability with DCGM Beta

GPU monitoring via Admin Access + Prometheus

DCGM (NVIDIA Data Center GPU Manager) is a monitoring tool for NVIDIA GPUs. It's usable with the **same** `adminAccess` mechanism — no privileged / hostPath needed.

```yaml
# ResourceClaim for monitoring
spec:
  devices:
    requests:
    - name: monitor-gpu
      exactly:
        deviceClassName: gpu-admin.nvidia.com
        adminAccess: true
```

```
curl localhost:9400/metrics | grep DCGM_FI_DEV
```

Key Metric	What it tells you
`DCGM_FI_DEV_GPU_TEMP`	GPU temperature (health)
`DCGM_FI_DEV_FB_USED`	How much VRAM is consumed?
`DCGM_FI_DEV_POWER_USAGE`	Power draw (capacity planning)

⚡ Live Demo: Admin Access + Observability

Module 8

```
# Admin Access: bypass GPU exclusivity for debugging
./scripts/phase2/run-module8-admin-access.sh
# Observability: deploy DCGM Exporter and verify Prometheus metrics
./scripts/phase2/run-module8-observability.sh
```

What to watch for:

1. `pod-gpu-1` Running (GPU occupied), `pod-gpu-2` Pending (no devices)
2. `pod-admin` Running despite GPU being "fully allocated", with `nvidia-smi -L` works inside
3. DCGM exporter runs via **adminAccess ResourceClaim** (no privileged, no hostPath)
4. GPU temperature and power metrics on port 9400

Module 9: Resilience

Your team is training a 70B-parameter LLM on 8x A100s.
It's been running for **29 days**. The DRA driver needs a version upgrade.

During the upgrade, the driver Pod restarts for 5 seconds.

Does the training crash? → **No.** ✅

Module 9: Why Running Pods Survive

CDI = Static Injection (not continuous proxy)

```
1. Plugin writes CDI spec       ← one-time job
2. containerd injects devices   ← reads CDI, mounts /dev/nvidia0
3. Container runs on GPU        ← direct PCIe bus communication
4. Plugin can now die           ← no impact on step 3
```

Docker analogy: After `docker run --gpus all` starts your container, the Docker daemon can be restarted without killing the container. The GPU connection is direct — the daemon was just the matchmaker.

The Blast Radius of DRA Components

Three failure layers:

Component Killed	Running Pods?	New Pods?
Controller (brain)	✅ Survive	✅ Resume after respawn
Kubelet Plugin (hands)	✅ Survive	❌ Wait for restart
MPS Daemon (runtime)	❌ **Fatal crash**	✅ After daemon restarts

⚡ Live Demo: Chaos Engineering

Module 9

```
# Experiment 1: Kill Controller → workload survives
./scripts/phase2/run-module9-resilience.sh
# Experiment 2: Kill Kubelet Plugin → workload survives
./scripts/phase2/run-module9-resilience-driver.sh
# Experiment 3: Kill MPS Daemon → workload dies (expected)
./scripts/phase2/run-module9-resilience-mps.sh
```

The highlight:

- Experiment 1: Watch `pod-resilience` stay `Running` after Controller deletion
- Experiment 3: Watch `nvidia-smi -L` fail inside the victim Pod

Architecture Decision Guide

```
SLA Guarantee? ── NO ──> Budget-sensitive? ── YES ──> DRA-managed MPS (M4)
      │                                     └── NO  ──> Full GPU (Exclusive)
      │
      └─ YES ──> Multiple Teams on same slice? ── YES ──> MIG + MPS (Hybrid)
                                                  └── NO  ──> MIG Only (Exclusive)
```

Quick reference:

Strategy	Isolation	Flexibility	Admin Access	Resilience
Full GPU	N/A	Low	✅ adminAccess	✅ CDI decoupling
MPS (DRA-managed)	Soft	High	✅ adminAccess	❌ MPS Shared Fate
MIG Only	Hard	Low	✅ adminAccess	✅ CDI decoupling
MIG + MPS	Hard + Soft	High	✅ adminAccess	✅ between slices, ❌ within slice

Summary: What We Built Today

Module	What	Key Takeaway
M0–M3	Kind + DRA Driver + First Pod	DRA replaces Device Plugin with structured allocation
M4–M5	DRA-managed MPS	Multiple Pods share one GPU
M6	vLLM on MPS	Real-world inference: Limitation impact on latency/throughput
M7	Consumable Capacity	(Alpha) API ready, driver not yet (Opaque limitation)
M8	Admin Access + DCGM	(Beta) Debug fully-allocated nodes + Prometheus metrics
M9	Resilience (Chaos)	Controller/Plugin crash = safe. MPS crash = fatal.
M10.1–10.5	MIG x DRA	CEL selectors + OOM isolation proof + MIG x MPS + silicon-to-Pod audit
M11	Stress & Boundary	Oversubscription + dual-MIG MPS + asymmetric sharing

Key Takeaways

1. DRA makes the Scheduler hardware-aware

No more blind GPU counting. The Scheduler reads device attributes and capacities.

2. MPS sharing requires zero special Pod config

Module 4: declare `sharing.strategy: MPS` in the Claim, driver handles everything.

3. MPS = maximize utilization, MIG = guarantee SLA

Combine them for the best of both worlds. Each MIG slice is an independent failure domain.

Key Takeaways (Cont.)

4. Admin Access + DCGM = Day 2 readiness

`adminAccess: true` bypasses exclusivity. DCGM gives you Prometheus metrics. Both are essential.

5. CDI decoupling = resilience by design

Driver crash ≠ workload crash. But MPS daemon crash = workload crash. Know the tradeoff.

6. DRA scheduler ≠ capacity enforcer

M11: 3×50% thread on 1 MIG all succeed; 3×8GiB on 20GiB MIG → 2 win, 1 OOM. Oversubscription is allowed; enforcement is at CUDA runtime, not scheduler. (Future: KEP-5075 Consumable Capacity)

Resources

Workshop Repository

All scripts used today: `github.com/Shiritai/k8s-dra-workshop`

Run Everything

```
# Full pipeline
./run_all.sh
# Individual phases
./scripts/phase1/run-module4-mps-basics.sh
./scripts/phase2/run-module8-admin-access.sh
...
# MIG mode toggle
sudo ./scripts/common/mig-reconfig.sh mig
sudo ./scripts/common/mig-reconfig.sh gpu
```

References

- K8s DRA Documentation
- NVIDIA DRA Driver
- MIG User Guide
- CDI Specification
- K8s DRA Workshop

Thank You

Questions?

Yang Tzu-Ching (Shiritai)
github.com/Shiritai/k8s-dra-workshop

All demos runnable at: dra-workshop/scripts